If it's not SBOM, then what? How Italian Practitioners Manage the Software Supply Chain

Sabato Nocera*, Massimiliano Di Penta†, Rita Francese*, Simone Romano*, and Giuseppe Scanniello*
*University of Salerno, Fisciano, Italy
†University of Sannio, Benevento, Italy
email: snocera@unisa.it, dipenta@unisannio.it, {francese, siromano, gscanniello}@unisa.it

2024 IEEE International Conference on Software Maintenance and Evolution (ICSME) | 979-8-3503-9568-6/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICSME58944.2024.00077

***Abstract*—A *Software Bill of Materials* (*SBOM*) describes, in a structured, machine-readable format, the open-source and proprietary components that constitute a software product, including their licenses, versions, vendors, vulnerabilities, and dependency relationships. SBOMs enable practitioners to gain visibility into the software supply chain and monitor any risks associated with software security, licensing, and more. In this industry paper, we present the findings of 10 semi-structured interviews with practitioners with different roles in six different software companies operating in Italy, some of which being very large multinationals. The gathered information indicates that the adoption of SBOMs is low, yet the attention of the software industry to software supply chain-related challenges is high. A possible reason behind this outcome is that the software industry has limited knowledge of SBOMs and software supply chain regulations. Although some participants showed a growing interest in SBOMs, the Italian software industry seems to respond less promptly to this technology. We plan to use these results and those from past research to design a survey with practitioners to have a complete picture of SBOM usage in the software industry.**

***Index Terms*—Software Bill of Materials, SBOM, Software Supply Chain, Interviews**

I. Introduction

A software supply chain consists of anything needed (*e.g.,* components and processes) to develop and deliver a software product. The software supply chain and its security have been recognized as a top priority by both the European Union (ENISA's *Guidelines for Securing the Internet of Things* [12] and *Cyber Resilience Act*, not yet enacted, proposed by the European Commission [11]) and the United States Government (President Biden's *Executive Order 14028 on Improving the Nation's Cybersecurity* [16]).

Software supply chain attacks involve the exploitation of vulnerabilities in components on which a software product depends. By leveraging a more mature and informed risk-based process, software producers can more proactively evaluate and remediate security risks in the software supply chain, thereby improving customers' and users' trust.

Software vendors often create products by assembling open-source and proprietary software components. An SBOM describes the inventory of components used in a software product [38]. To some extent, it is analogous to a list of ingredients on food packaging where the consumer consults its label to avoid that it may cause allergies. Even better, the presence of an SBOM in a software release can be compared to the presence of the "organic logo" [34] in food products, which allows one to follow the whole production chain. An SBOM is useful both to builders and buyers of software. On the one hand, builders often leverage available open-source and third-party software components to create a product; an SBOM allows them to make sure those components are up to date and to respond to new vulnerabilities. On the other hand, buyers use SBOMs to perform vulnerability or license analyses, both of which can be used to evaluate risk in a product.

The United States Government was the first to show interest in establishing guidelines to facilitate the adoption of SBOM. In particular, President Biden's Executive Order 14028 mandated the United States Department of Commerce and NTIA to publish the minimum elements for SBOMs [38].

While SBOMs are expected to shortly become the *de-facto* standard for any organization that develops or maintains software products in both industrial and open-source contexts, its adoption is still limited, as shown by recent studies [7], [19], [24], [32], [35], [40], [41]. This is due to several factors, including limited knowledge, lack of regulations, and limited tool support, especially for SBOM consumption (*e.g.,* tools to check if a software is using vulnerable components).

This paper presents the findings of semi-structured interviews with 10 practitioners from software companies operating in Italy. Some of these companies are very large multinationals. The results represent one of the outcomes of the Italian research project MSR4SBOM (Mining Software Repositories for enhanced Software Bills of Materials) [22], [30]. This project has been financially supported by the European Union NEXTGenerationEU project and by the Italian Ministry of the University and Research MUR, and its main goal is to deliver a framework that analyzes the content of software repositories and SBOMs to provide context-sensitive recommendations.

The mining study results suggested that, although the adoption of SBOMs is low, it has an increasing trend. Nevertheless, the results of the interviews presented in this paper indicated that the adoption of SBOMs is low, yet the interest of the software industry in the software supply chain is high. A possible reason behind this outcome is that the software industry has limited knowledge of SBOMs and software supply chain regulations. Although some participants showed a growing interest in SBOMs, the software industry seems to respond less promptly to this technology. The results of the research presented in this paper try to better understand the level of

2576-3148/24/$31.00 ©2024 IEEE
DOI 10.1109/ICSME58944.2024.00077

Authorized licensed use limited to: University Bern. Downloaded on April 20,2026 at 14:37:43 UTC from IEEE Xplore. Restrictions apply.

usage of SBOMs by practitioners, even if only multinational software companies operating in Italy took part in the study, and to promote this emerging technology in the industry.

In the following, Section II provides some background information about SBOMs and examines related work. Section III presents the study design. The results are reported in Section IV, along with possible practical recommendations. In Section V, we present the limitations of our study. Final remarks conclude the paper and outline directions for future work in Section VI.

II. Backgrounds and Related Work

In this section, we first provide some background notions about software supply chains and SBOMs. Then, we discuss related literature about SBOMs and software supply chains.

A. Software Supply Chains and Software Bills of Materials

The software supply chain and its security have been recognized as a top priority by both the European Union (ENISA's *Guidelines for Securing the Internet of Things* [12] and *Cyber Resilience Act*, not yet enacted, proposed by the European Commission [11]) and the United States Government (President Biden's *Executive Order 14028 on Improving the Nation's Cybersecurity* [16]). Despite receiving notable policy attention, the software industry has yet to implement SBOMs fully [41]. In this respect, the United States Government was the first authority to show interest in establishing guidelines to facilitate the adoption of SBOM. President Biden's Executive Order 14028 mandated the United States Department of Commerce and NTIA to publish the minimum elements and recommended data fields for SBOMs [38]. SBOM generation should produce a comprehensive and accurate list of all the components a software product depends on, which might vary depending on the stage of software development (*e.g.,* considering pre- vs. post-build artifacts) [23]. Manual SBOM generation is impractical due to the large number of dependencies that a software product can have. Therefore, it is necessary to rely on automated tools, possibly integrated into the development pipeline. For example, there are *Maven* plugins to be declared in `pom.xml` files for Java projects which allow the automatic SBOM generation, based on dependencies listed in the `pom.xml` or in general in build/lock files.

To make SBOMs machine-readable and then allow their consumption by other tools, different standards were defined. An SBOM standard is a machine-readable schema conceived to provide a uniform structure for describing the composition of a software product so that the SBOM can be consumed by other tools (*e.g.,* vulnerability or license scanners). As pointed out by Xia *et al.* [40], the most commonly adopted standards are SPDX [36] and CycloneDX [27]. SPDX is an open standard with origins in the *Linux Foundation*. Initially conceived to improve software license compatibility compliance [17], [28], SPDX was then extended to support further use cases like those related to security. In 2021, SPDX was approved as an ISO/IEC standard [14]. Another open standard for SBOMs is CycloneDX, developed by OWASP. Similar to SPDX, it supports different use cases, from software license compliance to security-related ones. The open nature of both SPDX and CycloneDX and the availability of software tools to generate SBOMs complying with these standards should ease their adoption. Our investigation goes in the direction of understanding if this assumption holds in open-source projects hosted on GitHub.

Although SBOM standards allow storing vulnerability information inside SBOMs, the United States Department of Commerce and NTIA suggest storing vulnerability information in separate data structures linked to SBOMs [38]. This is because vulnerability information is dynamic and evolves—a software product previously deemed not secure may become insecure as new vulnerabilities are discovered. As such, vulnerability information in SBOMs cannot be assumed to be complete and up-to-date unless very specific conditions and processes are in place [38].

B. Empirical Studies on Software Bills of Material

Linux Foundation surveyed SBOM's readiness in 2022 [35] on 412 worldwide organizations. The results of this survey showed gaps in familiarity with, production planning for, and consumption of SBOMs. For example, 40% of the organizations were unclear on whether the industry is committed to mandating SBOMs, raising concerns about whether the software industry is abiding by President Biden's Executive Order 14028 [16].

Stalknaker *et al.* [32] conducted a study combining surveys and interviews. This study investigated the challenges behind SBOM adoption for five different groups of stakeholders (practitioners familiar with SBOMs, contributors to critical open source projects, experts of Artificial Intelligence (AI) and Machine Learning (ML), developers of cyber-physical systems, and legal experts), and the ways such challenges could be circumvented. The results of their study, which involved 138 participants, identified 12 different challenges and possible solutions to address them. Important challenges are related to the need to have multi-dimensional SBOM specifications, improving SBOM-related tools, aiding SBOM verification/consumption, and (iv) favoring SBOM adoption, so far limited. Similarly, Xia *et al.* [40] conducted 17 interviews and a survey with 65 respondents to investigate *(i)* the adoption of SBOM in practice, *(ii)* the SBOM tool support, and *(iii)* the main concerns related to the use of SBOMs. Based on the collected results, the authors derived 26 statements describing the SBOM adoption and its challenges. Those are related, for example, to the still limited awareness, the need for better incentives in using SBOMs, a limited consensus on what should be the content of SBOMs, and the presence of sub-optimal and non-standardized tools, especially for what concerns the support for handling vulnerabilities.

Kloeg *et al.* [19] conducted semi-structured interviews whose goal was to analyze business stakeholder groups directly involved in SBOM production and consumption to determine which groups can drive or inhibit SBOM adoption and why. The gathered data suggested that system integrators

Authorized licensed use limited to: University Bern. Downloaded on April 20,2026 at 14:37:43 UTC from IEEE Xplore. Restrictions apply.

and software vendors had the highest SBOM adoption potential, whereas business-to-business customers and individual developers had the lowest motivation.

Chaora *et al.* [7] performed an ethnographic study analyzing discourse from public meetings during SBOM community engagement meetings facilitated by the Cybersecurity & Infrastructure Security Agency (CISA) in the United States Department of Homeland Security. To promote SBOM adoption, the authors foresee regulations and a public-recognized agency that audits and validates SBOMs submitted to a common infrastructure. The authors also foresee that SBOMs may face other challenges, such as supervising AI.

Our study looks at the adoption of SBOMs from a different perspective than those by Xia *et al.* [40], Chaora *et al.* [7], Stalknaker *et al.* [32], and Kloeg *et al.* [19]—*i.e.,* we gathered information on the adoption of software supply chain tools and SBOMs in the software industry. In addition, we aimed to understand how the software industry deals with problems related to the software supply chain like (i) the production and distribution of dependency inventories, (ii) the management of vulnerabilities and bugs, (iii) deprecated and outdated dependencies, (iv) the management of licenses, and (v) software provenance. Therefore, our study is complementary to these studies aiming to increase our body knowledge on SBOM adoption in the software industry (although our study is limited to software companies operating in Italy) and to delineate, on the basis of past research, possible differences between the software industry and open-source.

C. Empirical Studies on Software Supply Chains

Other literature is not directly related to SBOM, yet they study the challenges and risks arising from software supply chains. These risks can be, indeed, mitigated through the adoption of SBOMs. For example, Levy [21] asserts that open-source software can be more difficult to secure than proprietary software because of its development environment, which is more complex to a large extent. For example, organizations like Linux package the software commonly accepting code from others and bundling it with other packages. The resulting software is then distributed through websites. This complex web of interrelated parties creates several attack points in the software supply chain. The author highlights the need for technologies to secure supply chains of open-source software.

To enable sharing among industry practitioners having practical experiences and challenges with software supply chain security, Enck and Williams [10] conducted two industry and one government software summits (for a total of 30 organizations across the three summits). The result of these summits is a list of five challenges in software supply chain security. Among these five top challenges, there is *leveraging SBOM for security*. While participants had widely divergent opinions, they agree that efforts at establishing standards and requirements for SBOMs have the potential to lay the groundwork for innovative security enhancements.

Lamb and Zacchiroli [20] examine Reproducible-Builds (R-B) from the perspective of practitioners. R-B is an approach that can determine whether generated binaries correspond to a given source code. The main idea is that if we are confident that building a given source tree always generates bit-for-bit identical results, we can trust these artifacts. The results of their study indicate that uncontrolled build inputs occur when tool chains allow the build process to be affected by the surrounding environment.

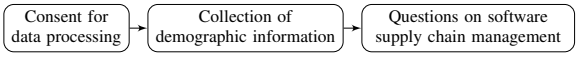

Fig. 1. High-level view of the interview script.

TABLE I
QUESTIONS FOR COLLECTING DEMOGRAPHICS INFORMATION.

Questions
1. What is your educational qualification?
2. How long have you worked as a software practitioner?
3. What company do you work for, and in what division?
4. What is your role in the company?
5. How many years have you been in that role?
6. What domain do your past and present projects fit under? Who commissioned them?
7. Do you develop software for the Italian public administration? Have they placed any constraints on software dependencies?
8. Do you develop software for EU clients? Have they placed any constraints on software dependencies?
9. Do you develop software for US clients? Have they placed any constraints on software dependencies?

III. INTERVIEW STUDY DESIGN

In this section, we describe the design of our study.

A. Study Protocol

The study has been conducted in the form of semi-structured interviews, fitting its exploratory nature. Moreover, semi-structured interviews allow asking for clarifications and/or details if needed or in case a question is not clear to the participant [29], [31].

We recruited ten practitioners from six companies in our contact network. Those six companies are national or the Italian division of multinationals. Due to internal policies and considering the nature of this study, the companies preferred not to have their names disclosed. Four practitioners were interviewed in person, while, because of logistic and distance reasons, the other six were interviewed remotely (*i.e.,* through a Microsoft Teams video call). Each interview session involved the same interviewer and one participant at a time. We obtained the participants' consent to audio-record the session at the beginning of each interview session. We also informed the participants that the gathered data would be treated confidentially and only for research purposes. Each interview lasted between 20 and 62 minutes (on average, 33 minutes). Since we conducted semi-structured interviews, the questions were not necessarily asked in the order presented in the interview script, as guidelines on qualitative studies outline [29], [31]. The reason is that, depending on the evolution of the conversation, some questions could have been addressed earlier than others.

Authorized licensed use limited to: University Bern. Downloaded on April 20,2026 at 14:37:43 UTC from IEEE Xplore. Restrictions apply.

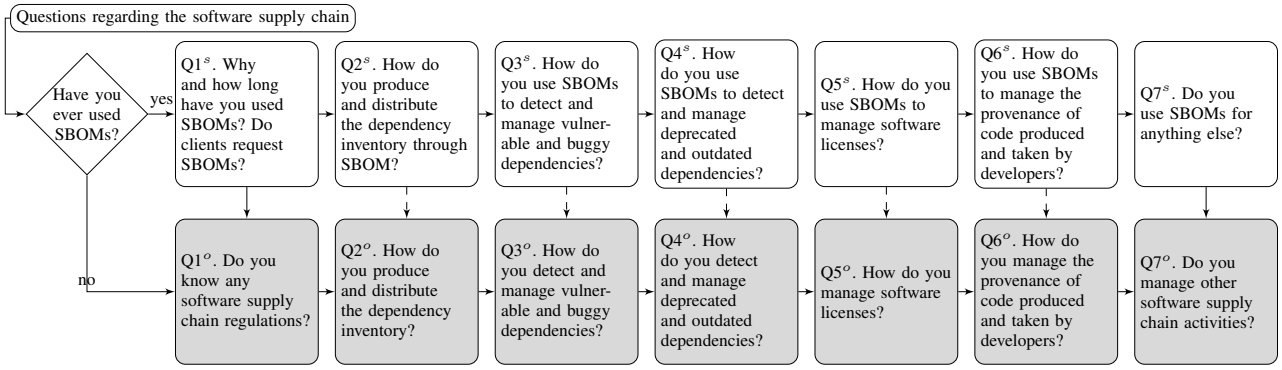

Fig. 2. Questions for collecting information on software supply chain management.

TABLE II
RATIONALE OF QUESTIONS ON SOFTWARE SUPPLY CHAIN MANAGEMENT.

Questions
Q1. Identify software supply regulations followed by companies
Q2. Comprehend if and how dependency inventory is managed (*e.g.,* technologies like SBOMs, tools like Microsoft Excel)
Q3. Comprehend if and how vulnerable and buggy dependencies are managed
Q4. Comprehend if and how deprecated and outdated dependencies are managed
Q5. Comprehend if and how software licenses are monitored and verified
Q6. Comprehend if and how code provenance is managed (*e.g.,* code taken from Q&A forums, ownership of code written by company's developers)
Q7. Identify other software supply chain activities (that can eventually be supported through SBOMs)

Therefore, semi-structured interviews enabled improvisation and exploration of the investigated phenomenon.

The interviews followed a script to ensure that all relevant topics were covered [29], [31]. Fig. 1 depicts the high-level view of such a script. At the beginning of each interview, the interviewer asked the participant for consent to data processing. Then, the interviewer asked demographic questions regarding the participants' education, practitioner experience, and the types of projects/clients they worked on/for. These questions are shown in Table I and are aimed at characterizing the study context better. For instance, different projects and clients may require different requirements [25]. The last part of the interview involved questions on how the participants' companies managed the software supply chain. These questions are shown in Fig. 2, while the rationale behind each question is provided in Table II. The questions we asked to the participants were different, based on whether or not they had used SBOMs in their company. As shown in Fig. 2, if the participant had used SBOMs, the interviewer would have asked the questions reported in the *white* rounded boxes and denoted as Qx^s. If the participant had never used SBOMs, the interviewer would have asked only the questions reported in the *gray* rounded boxes and denoted as Qx^o – other. Note that if the participant did not answer a question denoted by the '^{s}' symbol (*e.g.,* because it did not use SBOMs for the reported activity), the matching question marked with the '^{o}' symbol and indicated by the dashed arrow would have been asked, *i.e.,* to gather data on if there was an alternative to SBOM to address the given issue. Note that the participants who had used SBOMs were also asked the first and the last questions in the gray rounded boxes to understand better how their companies managed the software supply chain; in this case, the arrows connecting the white questions to the gray ones are not dashed but solid.

B. Study Participants

Table III reports the participants' demographics resulting from the answers to the questions in Table I. We refer to each participant by an ID (from P1 to P10) to preserve their anonymity. The IDs of the participants have been established according to the order in which we interviewed them. We also had to guarantee the anonymity of the company where the participants worked at the time of the interview. To that end, each company is indicated with a letter (from *a* to *f*).

As participants in this research, we involved companies from our contact network. We asked these companies to recruit possible participants with knowledge of software supply chain and how it is handled within their own company. The companies shared an internal notice with the employees satisfying these criteria. The note described the aims of the study and the types of activities proposed. At the end of this process, we had two IT managers, five team leaders, a chief executive officer, and two solutions architects. Their experience as practitioners was in between 4.5 years (P7) and 30 years (P6). The average experience of the practitioners was about 18 years.

The participants were from six different companies, all being either Italian or an Italian branch of a multinational. The participants of the same company had different roles and/or experience, *e.g.,* they could have worked on different projects or with different customers. In particular, P1 and P2 worked for the Italian branch of the same multinational company (*a*), which operates in the field of cybersecurity and personal data protection and has more than 500 employees (15,000 worldwide). P3 was the chief executive officer of an Italian company (*b*), which operates in the field of smart

Authorized licensed use limited to: University Bern. Downloaded on April 20,2026 at 14:37:43 UTC from IEEE Xplore. Restrictions apply.

TABLE III
STUDY PARTICIPANTS' DEMOGRAPHICS.

ID	Experience as Practitioner	Company Division	Role	Job Tenure	Project domain	Clients	Dependency Constraints
P1[a]	17 years	Security	IT Manager	6 years	Security testing, vulnerability management, and network security	Banking, finance, telecommunications, and energy in Italy	None
P2[a]	5 years	Security	Team Leader	5 months	Critical systems	Public administration, banking, and insurance in Italy	None
P3[b]	30 years	Top management	Chief Executive Officer	9 years	Software migration, e-learning, IoT, and smart buildings	Public Administration in Italy and other organizations in Italy, the EU, and the US	None
P4[c]	26 years	Software Development	Team Leader	2 years	Legacy software for retail management	Fashion industry in Italy	None
P5[c]	22 years	Software Development	Team Leader	5 years	Logistics, research, IoT, and blockchain	Organizations in the EU	None
P6[d]	30 years	Security	Solutions Architect	5 years	Troubleshooting and automation	Telecommunications companies worldwide	Laws of specific countries
P7[d]	4 and a half years	Software Development	Team Leader	11 months	Data engineering	Telecommunications companies worldwide	None
P8[e]	21 years	Application and Infrastructure Services	Solutions Architect	5 years	Custom application development	Public administrations, banking, insurance, *etc.* in Italy	Public administrations mandating the use of specific third-party components or frameworks
P9[f]	20 years	Top management	IT Manager	7 months	Banking, finance, and billing	Public administrations in Italy and other organizations in Italy and the EU	The company has a list of allowed (and not allowed) third-party components and technologies for them and their vendors
P10[d]	8 years	IT Monitoring and automation	Team Leader	2 years	Network and infrastructure monitoring	Telecommunications companies worldwide	They must comply with clients' software supply chain security requirements and the laws of specific countries

Each letter (*a*, *b*, *c*, *d*, *e*, and *f*) indicates one of the six companies to which the participants belonged; thus, participants with the same latter work for the same company.

TABLE IV
INITIAL TEMPLATE OF CODES.

Theme	Focus
1) Regulations	Regulations for managing the software supply chain
2) Dependency Inventory	Production and distribution of the dependency inventory
3) Vulnerable Dependencies	Detection and management of vulnerable dependencies
4) Buggy Dependencies	Detection and management of buggy dependencies
5) Deprecated Dependencies	Detection and management of deprecated dependencies
6) Licenses	Software license management
7) Provenance	Source code provenance

TABLE V
FINAL TEMPLATE OF CODES.

Theme	Sub-theme
1) Regulations	a) Security b) Release
2) Dependency Inventory	c) Software documentation d) Package managers e) SBOM
3) Dependency Update	f) Project type g) Bulletins h) CI/CD pipelines
4) Licenses	i) License Compliance
5) Provenance	j) Code produced by developers k) Code taken by developers

and sustainable Internet-of-Things (IoT) environments and has about ten employees. P4 and P5 worked for the same branch of an Italian company (*c*), which mainly develops software services and solutions and has nearly 200 employees. P6, P7, and P10 worked for the Italian branch of the same multinational company (*d*), although in different divisions (*i.e.,* security, software development, and IT monitoring/automation, respectively). This company's main business is the sale of infrastructure, software, and services in the information and communications technology field. The Italian branch of this company has nearly 2,000 employees (100,000 worldwide). P8 worked for the Italian branch of a multinational company (*e*), which mainly develops software services and solutions and has more than 5,000 employees (almost 200,000 worldwide). Lastly, P9 worked for an Italian company (*f*), which operates in the finance sector and has about 1,500 employees.

C. Data analysis

After transcribing the recordings of each interview, the first author analyzed the transcripts using a thematic analysis approach, namely template analysis [18]. Template analysis is a flexible and fast data analysis approach that allows the researchers to develop a list of codes, each identifying a theme within the transcripts. The codes are arranged in a template whose structure is usually hierarchical. Such structure highlights the relationships among the identified themes. In template analysis, the researchers analyze the transcripts using an initial template of codes. The purpose is to delimit

Authorized licensed use limited to: University Bern. Downloaded on April 20,2026 at 14:37:43 UTC from IEEE Xplore. Restrictions apply.

portions of text in transcripts related to the themes of interest. Following the recommendations of King [18], we developed our initial hierarchical template based on the used interview script (see Table IV). As customary, we revised the initial template during the analysis [18]. In Table V, we show the final template of codes. In particular, we aggregated the first-level codes *Vulnerable Dependencies*, *Buggy Dependencies*, and *Deprecated Dependencies* in a single first-level code, namely *Dependency Update*. Based on the participants' answers, vulnerable, buggy, and deprecated dependencies are detected, managed, and fixed similarly. In fact, a vulnerable dependency typically includes a (security) bug and becomes deprecated when the vulnerability-fixing version is released. Moreover, for each first-level code, we identified two or three sub-codes in the final template. We concluded the template analysis when all portions of transcripts relevant to the goal of our study were coded. To ease the thematic analysis of the transcripts, we used the ATLAS.ti [6] tool. This tool supports qualitative data analyses, including template analysis.

IV. Results

In this section, we present the findings emerging from the interview study. These findings are presented according to the main themes identified by the first-level codes of the final template shown in Table V. For each finding, we also provide a summary of such a finding together with some recommendations for practitioners.

A. Regulations

This theme includes three sub-themes related to software supply chain regulations, namely *security*, *artificial intelligence*, and *release*.

a) Security. As for software supply chain security regulations, three participants (*i.e.,* P2, P6, P9) heard about the Cyber Resilience Act. In particular, P2 heard about it at a conference, while P6 and P9 heard about it through their company communication channels (*e.g.,* newsletters). In the same way, P6 and P9 also heard about Executive Order 14028 on Improving the Nation's Cybersecurity. None of the participants knew any details about such regulations. Additionally, P2 mentioned two other security regulations their company follows and for which they are certified, namely *DORA* and *TIBER-EU*. They mentioned how such certifications are necessary to work for certain customers. However, according to P2, such regulations are more concerned with security testing than with the software supply chain. Therefore, they do not regulate the use of SBOMs.

P9 and P10 knew and followed some software supply chain security regulations despite not knowing the names of any of them. More in detail, P9 stated that their company must comply with precise software supply chain security regulations to operate in such kinds of domains (*i.e.,* banking, finance, and billing). Such regulations involve the handling of *(i)* third-party components that must not be included among software dependencies and *(ii)* services the company provides to its clients (*e.g.,* verifying software security or providing security certifications at any time). The P9's company follows such regulations to operate within the bank domain. P10 was aware of the existence of software supply chain regulations concerning SBOMs. P10 was the only participant who knew—and used, to some extent—SBOMs. In this respect, P10 mentioned that SBOM has been a hot topic for the past year and is among the standards followed by their company. Note that P6 and P7 belonged to the same company as P10 but did not know SBOM. We conjecture that the company has been incrementally and experimentally introducing SBOMs in their company by conducting pilots in different divisions. While we discussed this conjecture with P10, this participant did not have a complete view of the company's strategies and, therefore, was not able to confirm it.

b) Release. P3 pointed out that each country may have different regulations regarding the deployed software. For example, these regulations could limit the kinds of features software provides in each of these countries (*i.e.,* a functionality could be available in one country while it cannot be released in another country). According to P3, the release sub-theme concerns third-party components integrated into the software under development. Therefore, software vendors should always pay attention to the components of software. In this context, it is evident how beneficial a tool for software supply chain support can be.

Summary: Despite the interviewed practitioners knowing little about SBOM and its related regulations, they tend to follow good practices concerning supply chain management. More advanced practices and regulations are almost unknown.

Recommendation: The software industry should not neglect software supply chain regulations and keep their employees informed about them. A better knowledge transfer about software supply chain practices, SBOMs, and, above all, their implications in specific domains such as service-oriented systems is highly desirable. To this end, as other companies have done, informative communications can be made through dedicated channels (such as the company's newsletters).

B. Dependency inventory

This theme includes two sub-themes related to the production and distribution of the inventory of software dependencies. In this respect, we identified the following three sub-themes in the participant transcripts: *software documentation*, *package managers*, and *SBOMs*.

c) Software documentation. P1, P3, and P9 relied on software documentation to produce and distribute the inventory of software dependencies. According to P1, such documentation is based mainly on UML diagrams, *e.g.,* component or class diagrams. However, UML-based

Authorized licensed use limited to: University Bern. Downloaded on April 20,2026 at 14:37:43 UTC from IEEE Xplore. Restrictions apply.

documentation does not describe fine-grained software dependencies (*e.g.,* libraries). P3 used spreadsheets to report information on software dependencies (*e.g.,* version, licenses). This information is entered manually. Therefore, updating it can be prone to errors and not match the dependencies that are really present in the source code. P9 produces documentation on dependency inventory through Archi, a modeling toolkit for describing, analyzing, and visualizing architectures within and across business domains [5]. This toolkit is also capable of capturing dependencies between software components. As part of the software documentation, the dependency inventory is created and then distributed to the clients who have the software as part of the contract.

d) Package managers. P5 and P8 relied only on package managers (*e.g.,* `Maven`) for the dependency inventory. Developers can declare the software dependencies in the package manager files (*e.g.,* `pom.xml`). Therefore, package manager files represent the inventory of software dependencies and are distributed to the client who owns the software as part of the software itself. However, the client must have enough technical knowledge to understand the information reported in package manager files.

e) SBOM. Only one of the six companies used SBOMs to produce and distribute the dependency inventory. Indeed, based on the interviews we performed, we noted that among the three participants of that company, namely P6, P7, and P10, only one was aware of using SBOMs, namely P10. P6 referred to SBOMs as general "reports" of dependency inventory. P7 stated to declare software dependencies within package manager files. P7 also reported information on such dependencies in an external text file that was handed over to the DevOps team of the company (of which P10 is part). As a team leader of the DevOps team, P10 was responsible for processes involving dependency management, including using SBOMs. This difference among P6, P7, and P10 about the use of SBOMs as inventory could be because the company is very huge and they belong to different divisions. That is, any division could have a limited view of other divisions and each division introduces new technologies adopting different timing and strategies.

P10 described the process followed by their company to produce and distribute SBOMs. Specifically, the company sets up its Continuous Integration and Delivery (CI/CD) pipelines to produce SBOMs periodically or at the occurrence of an event. As P10 reported, their company uses three SBOM generation tools, namely Syft [3], Trivy [4], and XRay [15] (all supporting both SPDX [36] and CycloneDX [27] SBOM formats). Such tools extract software dependencies by analyzing package manager files. Then, they retrieve further information (on transitive dependencies, suppliers, *etc.*) from external data sources. The choice to use multiple tools is because different tools can produce different SBOMs, *i.e.,* different lists of dependencies, and, therefore, this increases the chance of identifying all dependencies. Related to that, P10 mentioned that SBOM generation tools could have recall and precision concerns. In this respect, P10 also told us that, in the past, the company used another SBOM generation tool, namely Anchore [1], which was abandoned due to the production of too many false positives. Since more than one SBOM generation tool is used, the company had to merge the information from multiple SBOMs into a single one. Moreover, to verify (and eventually integrate) the SBOM generation tools, single developers (such as P7) have to manually annotate each software dependency they introduce and its version. As previously mentioned, the developers write this information in an external textual file, which is handed over to the DevOps team. Lastly, once the SBOM is completed, the dependency inventory is declared within an in-house monitoring system. When a dependency issue emerges, all projects that have declared such a dependency in the in-house monitoring system are notified. P10 claimed that SBOMs are particularly useful for distributing the dependency inventory because they have different formats (*e.g.,* JSON) that can be manipulated through various tools. Thus, even those without specific technical skills can interact with the software dependency inventory.

Summary: SBOMs are effective in managing software dependency inventory at finer grain levels.
Recommendation: The software industry can take advantage of SBOM to document dependency information at different levels of granularity, including at the library level, which cannot be effectively represented by common software documentation, *e.g.,* UML diagrams. The use of multiple SBOM generators, as well as manual dependency annotation, can enhance the reliability of the provided dependency information. Last but not least, further work needs to be done to provide reliable, automated SBOM generation tools.

C. Dependency Updates

This theme includes three sub-themes related to detecting and managing software dependencies, namely *project type*, *security bulletins*, and *CI/CD pipelines*.

f) Project type. The strategy for managing the dependency update changes according to the project type. As stated by P5, clients who own legacy software may prefer not to update any dependencies as long as their software is working properly. P10 pointed out that software implemented with older technologies could be too difficult, if not impossible, to update. When dependency update is not possible, isolating the software (*e.g.,* through containers) is a way to limit the risk of security issues.

When the software requires evolutionary and/or corrective maintenance, practitioners may adopt a more proactive approach to dependency updates by keeping dependencies to third-party libraries always up-to-date at the latest

Authorized licensed use limited to: University Bern. Downloaded on April 20,2026 at 14:37:43 UTC from IEEE Xplore. Restrictions apply.

available version and resolving incompatibilities incrementally as soon as they occur. This is done to reduce the risk that the presence of several too-old, incompatible dependencies causes a hard-to-resolve building breakage.

g) Bulletins. P1 and P2 rely on bulletins to detect and manage dependencies, especially vulnerable ones. The information on vulnerable dependencies is reported in security bulletins. These bulletins are published daily/weekly by authoritative entities such as NIST, which also maintains the NVD and is responsible for publishing CVEs. Security bulletins may contain the remediation for the reported vulnerable dependencies (*e.g.,* changing the dependency version or applying a fix in the source code).

A Security Operation Center (SOC) may supervise the validation of software dependencies against the vulnerabilities disclosed in security bulletins. Each client who owns the software establishes with the SOC the level of service desired (*e.g.,* 24 hours a day, five days a week). This may also be done on single dependencies—*i.e.,* it is possible to retrieve and monitor all the security bulletins of a specific dependency.

h) CI/CD pipelines. P3, P8, and P10 pointed out CI/CD as an important part of the software supply chain. Software development may involve the production of several modules that, after being implemented, must be integrated. Each module can evolve independently but must always be able to integrate with the others, which could be quite complex. CI/CD pipelines are employed to verify the integration of the different components that constitute software. CI/CD pipelines can also verify the delivery/deployment of software in specific environments and according to specific technological constraints. To this end, CI/CD pipelines perform tasks periodically and/or at the occurrence of certain events (*e.g.,* pull requests, project milestones, or client requests), including security tasks. To perform all these tasks, CI/CD pipelines leverage different program analysis tools, such as those based on static analysis, Software Composition Analysis (SCA), and SBOM. It is possible to define certain criteria (*e.g.,* not using a particular version of a dependency) so that the CI/CD pipeline fails if such criteria are not met. SCA tools analyze the software to retrieve the list of the vulnerable and deprecated dependencies, along with their remediation, if any. Among the SCA tools, P1 declared to mainly use Fortify [26], a commercial product. In the past, they also used Checkmarx [8]. P8 recommended the use of Trivy as a SCA tool. As previously mentioned, Trivy also supports the production and management of SBOMs, although P8 stated not to know and use SBOMs. CI/CD pipelines may also involve SBOM production and consumption. Similar to SCA tools, SBOM consumption tools allow for checking vulnerable and deprecated dependencies from SBOMs. The company of P10 had the DevOps team analyze the SBOMs produced through Trivy and Grype [2]. Being part of the DevOps team, P10 defined the user stories to resolve the detected vulnerable and deprecated dependencies. P6 also pointed out the use of the Nessus [33] vulnerability scanner. Moreover, according to P6, the company's in-house monitoring system checks daily for any newly published CVE that refers to dependencies declared by any of its projects and eventually notifies them. More generally, such an in-house system checks for all dependencies that need to be updated, including deprecated ones.

Summary: Software dependencies may need to be updated due to issues within them, such as vulnerabilities. These dependencies can be detected by CI/CD processes and divulged by bulletins published by authoritative entities. Nevertheless, updating them may differ depending on the type of project and the technologies it uses.
Recommendation: Companies should integrate SCA or SBOM tools in their CI/CD pipelines to identify dependencies that need to be updated. SCA/SBOM tool producers should improve these to support legacy software.

D. Licenses

This theme includes only one sub-theme related to the management of software licenses, namely *license compliance*.

i) License Compliance. According to P7, some clients prefer employing (free) open-source dependencies to avoid incurring further costs. In addition, since the source code of open-source dependencies is available, clients could request modifying it according to their specific needs. This indicates a severe problem: software owners may not know the difference between open-source and free software. This is crucial because, according to P3, software owners are primarily responsible for license compliance.

A possible solution for managing license compliance is to (manually) track information about each license on a spreadsheet. There are also solutions that (partially) automate licensing management. For instance, P6's company mainly relies on XRay and Trivy tools that, however, simply identify dependencies along with their licenses, leaving the incompatibility identification to the practitioners. The reason for using multiple tools is to gather as much information (on licenses) as possible. However, different tools' outputs may conflict. P10 recalled that, in the past, it happened that XRay and Trivy produced different licenses for the same dependency. Therefore, they manually checked the incorrect licensing-related information and observed that XRay produced the incorrect output. They then had to open a ticket to the vendor of this tool reporting the detected problem. Lastly, through its in-house monitoring system, the same company checks for any software dependencies that could violate its company policies. The licensing incompatibility identification was a challenge for the company in which P6, P7, and P10 worked (they all worked in the same company), especially when third-party dependencies have several

Authorized licensed use limited to: University Bern. Downloaded on April 20,2026 at 14:37:43 UTC from IEEE Xplore. Restrictions apply.

different types of licenses. Although the company used SBOM-based technologies (which help with license management), such verification was still pretty manual and did not appear overly structured.

Summary: The management of software licenses is still pretty manual.
Recommendation: The software industry should adopt SBOMs to improve the awareness of the software licenses they adopt and automate their management.

E. Provenance

This theme includes two sub-themes related to the management of dependency provenance, namely *code produced by developers* and *code taken by developers*.

j) Code produced by developers. Most participants (P1, P4, P5, P6, P7, P8, P9, and P10) were aware of laws, including penal laws and company policies regulating code reuse by developers. These laws and policies mainly focused on code written by developers. According to them, code written by developers cannot generally be reused outside of the software it was written for. Some of these laws may also prohibit disclosing information about the software being developed. This concern is particularly relevant when developers write code for clients of their company, as they are the ones owning/commissioning software.

P1, P3, P4, P7, P10 felt it was still too difficult—if not impossible—to identify developers who reuse code illegally. They believed that developers' responsibility and the deterrent provided by laws/policies are practically the only means of reducing the risk of illegal source code reuse. P7 believed there might be source code whose reuse is not a concern. P7 brought the experience of implementing a graphics component that became a reusable library as an example. P7 did so because such a component does not involve sensitive code. P7 also indicated that source code reuse is a problem, especially for the back-end, since this code might involve more sensitive code than that of the front-end.

The only participant who was aware of tools to identify illegal code reuse was P9. This participant stated that their company has software that scans public Git repositories to verify that company code is not being outsourced and made public. Previously, the company discovered a developer who had illegally reused its source code. This was why this company equipped itself with such a tool.

k) Code taken by developers. Most participants (*i.e.,* P3, P4, P6, P7, P8, P9, P10) explicitly stated that their companies do not have guidelines for code taken from external sources, such as generative AI (*e.g.,* ChatGPT, GitHub Copilot) or Stack Overflow. As for generative AI, P3 and P5 heard about the *EU AI Act* [9] by reading online blogs; however, they knew nothing more detailed about regulations regarding the use of AI in software development. P7 believed that the ethics of individual developers play a key role in the use of generative AI. As soon as ChatGPT was released, the company emailed employees, warning them of the possible dangers of generative AI (*e.g.,* sharing sensitive data). P7 mostly used ChatGPT to generate code from scratch rather than modify existing code; in the latter case, P7 avoided providing sensitive information or complete code snippets. The P1 and P8 companies were working to regulate the use of generative AI. P1 explicitly stated that, to date, their company forbids the use of generative AI because it has not yet been regulated, and their clients do not require the use of such technology, but neither they forbid its usage. The results related to this sub-theme open an interesting point related to the software supply chain and generated source code.

It must be noted that currently SBOMs are primarily designed to describe dependencies at the level of software components/packages. However, this would not preclude the possibility of tracing artifacts at varying levels of granularity (e.g., files reused from other projects) and, when the technology will allow that, also trace the chain of AI-generated artifacts.

Software companies can take various measures to deter developers from reusing code illegally. Among them, participants reported awareness-raising actions (*e.g.,* training courses), signing non-disclosure agreements, and using secured equipment provided by the company. In the latter case, participants referred to computers that contain the software to work on and that are restricted in the actions developers can perform. For example, these mentioned computers do not allow for copying and pasting of code. They are also configured to prevent developers from accessing online resources that are not verified by the company.

P3, P9, and P10 believed using code from generative AI is not particularly dangerous, as their CI/CD pipelines would inspect the code and detect any concerns.

Summary: According to the participants' answers, 5 out of 6 companies prohibit developers from redistributing their code (*e.g.,* on hosting platforms like GitHub), while only one company forbids the use of generative AI in software development. Despite the adoption of such policies, it is still difficult to track and manage software provenance.
Recommendation: Our recommendation is to use SBOMs because they could be envisioned to support software provenance tracking and monitoring. To this end, the vendors of SBOM tools should integrate features for detecting the provenance of reused code, as well as identifying AI-generated code.

V. Threats to Validity

Threats to *internal validity* concern factors, internal to the study, that could have influenced our results. The sampling

Authorized licensed use limited to: University Bern. Downloaded on April 20,2026 at 14:37:43 UTC from IEEE Xplore. Restrictions apply.

method used in the interview study might have affected the results. Participants in the interview study might not have answered truthfully because they were scarcely motivated or afraid of being judged. To mitigate this threat, participation in the interview was voluntary because volunteers are generally more motivated [39]. This was why we asked the companies involved in the study to recruit participants in the interview through an internal notice with all their employees with knowledge of software supply chain (see Section III-B). We also informed the participants (before the interview took place) that the gathered data would be treated confidentially.

Investigators might unconsciously influence the results based on their expectations. We mitigated such a threat by involving more than one author in the analyses of the gathered data (*i.e.,* we applied investigator triangulation [37]).

Participants could not have understood themes or points. To mitigate this threat, we approached the themes top-down, starting with the general ones and moving to the specific ones.

The study did not concern sensitive information regarding companies and their employees. Therefore, confidentiality should not impact the validity of the results from our semi-structured interviews. In any case, we did our best to control this kind of threat while participants discussed the themes of interest of our study with the interviewer.

Threats to *external validity* concern the generalizability of our findings. The number of interviews might threaten the validity of the results. However, Guest *et al.* [13] observed that a sample of six interviews may be sufficient to derive meaningful themes and interpretations. Based on the analysis of the ten interviews, we believe we have reached a point of diminishing return [13] for which conducting more interviews is unlikely to provide more evidence. In addition, we plan to use the results of our study (together with those from past research shown in Section II-B and further discussed in relation to our outcomes in Section VI) to design a survey with other practitioners to have a complete picture of SBOM usage in the Italian software industry. The survey will be based on questionnaires and include quantitative questions to apply methodological triangulation [37]. In other words, we will assess the level of agreement among survey participants with the statements made by the participants and see whether there are any differences.

Finally, we recognize that our results may not hold outside the context where the study has been conducted. That is, since the results are based on evidence collected from participants with practitioners from six different software companies operating in Italy, we cannot claim that they will apply to all the software companies operating in Italy and around the world.

VI. Discussion and Conclusion

In this paper, we present the findings of semi-structured interviews with 10 industrial practitioners from software companies operating in Italy. Some of these companies are very large multinationals. The results of this study represent one of the outcomes of an Italian research project named MSR4SBOM [22], [30] (Mining Software Repositories for enhanced Software Bills of Materials). We conducted these semi-structured interviews in MSR4SBOM to understand the state of the practice of the software supply chain and the usage of SBOMs in the Italian software industry.

Our preliminary findings indicate that the adoption of SBOMs is low, yet the interest of the software industry in the software supply chain is high. A possible reason behind this outcome is that the software industry has limited knowledge of SBOMs and software supply chain regulations. Although some participants showed a growing interest in SBOMs, the software industry seems to respond less promptly to this technology.

In comparison with previous studies [32], [35], [40], mainly conducted in a US-based context or in a wider context, we found both commonalities and differences:

- The awareness about the importance of producing SBOM is quite low in Italy, if compared to what happens, for example, in the US. We conjecture this may also be due to the lack of strict regulations—similar to the US Executive Order for improving Nation's CyberSecurity [16]—that forces software companies (in the role of software producers) to produce SBOMs in certain circumstances. That being said, related EU Acts such as the Cyber Resilience Act [11], but also the EU AI Act [9] are posing the basis for better transparency and management of the software supply chain.
- One challenge clearly emerging from our study that confirms the previous finding is the limited availability of tools for SBOM production and consumption and, in general, for supply chain management. From our study, it emerges that in the Italian context, several activities related to dependency management and licensing compliance analysis are mostly performed manually or by combining multiple tools, often providing contradictory results. Therefore, the road toward reliable tools and their wider adoption in the industry—especially in the studied context, is still long. Our findings suggest that an intense technology transfer from academia to industry on supply chain and SBOM-related topics is highly desirable.

In future work, we plan to start from the obtained results and from those of previous, wider studies [32], [35], [40] to design a survey with practitioners and have a complete picture of the SBOM usage in the Italian software industry. The results of such a study will aim to completely enhance the primary objective of the MSR4SBOM (Mining Software Repositories for enhanced Software Bills of Materials) project, namely understanding and favoring the usage of SBOMs.

Acknowledgments

This project has been financially supported by the European Union NEXTGenerationEU project and by the Italian Ministry of the University and Research MUR, a Research Projects of Significant National Interest (PRIN) 2022 PNRR, project n. D53D23017310001 entitled 'Mining Software Repositories for enhanced Software Bills of Materials (MSR4SBOM)'.

Authorized licensed use limited to: University Bern. Downloaded on April 20,2026 at 14:37:43 UTC from IEEE Xplore. Restrictions apply.

References

[1] anchore. (2024) anchore. Last Accessed: June 26, 2024. [Online]. Available: https://github.com/anchore/anchore/

[2] ——. (2024) grype. Last Accessed: June 26, 2024. [Online]. Available: https://github.com/anchore/grype/

[3] ——. (2024) syft. Last Accessed: June 26, 2024. [Online]. Available: https://github.com/anchore/syft/

[4] Aqua Security Software. (2024) Trivy home - Trivy. Last Accessed: June 26, 2024. [Online]. Available: https://trivy.dev/

[5] Archi. (2024) Archi- open source ArchiMate modelling. Last Accessed: June 26, 2024. [Online]. Available: https://www.archimatetool.com/

[6] ATLAS.ti. (2024) Computer-assisted qualitative data analysis software. Last Accessed: June 26, 2024. [Online]. Available: https://atlasti.com/

[7] A. Chaora, N. Ensmenger, and L. J. Camp, "Discourse, challenges, and prospects around the adoption and dissemination of software bills of materials (SBOMs)," in *2023 IEEE International Symposium on Technology and Society (ISTAS)*, 2023, pp. 1–4.

[8] Checkmarx. (2024) Application security testing tool - Checkmarx Appsec solution. Last Accessed: June 26, 2024. [Online]. Available: https://checkmarx.com/

[9] Council of the European Union. (2024) Proposal for a regulation of the European Parliament and of the council laying down harmonised rules on artificial intelligence (Artificial Intelligence Act) and amending certain union legislative acts. Last Accessed: June 26, 2024. [Online]. Available: https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai

[10] W. Enck and L. Williams, "Top five challenges in software supply chain security: Observations from 30 industry and government organizations," *IEEE Security & Privacy*, vol. 20, no. 2, pp. 96–100, 2022.

[11] European Commission. (2022) EUR-Lex - 52022PC0454 - EN - EUR-Lex. Last Accessed: June 26, 2024. [Online]. Available: https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=celex%3A52022PC0454

[12] European Union Agency for Cybersecurity. (2020) Guidelines for securing the Internet of Things - ENISA. Last Accessed: June 26, 2024. [Online]. Available: https://www.enisa.europa.eu/publications/guidelines-for-securing-the-internet-of-things/

[13] G. Guest, A. Bunce, and L. Johnson, "How many interviews are enough? an experiment with data saturation and variability," *Field methods*, vol. 18, no. 1, pp. 59–82, 2006.

[14] ISO/IEC. (2021) ISO/IEC 5962:2021 - Information technology — SPDX® specification V2.2.1. Last Accessed: June 26, 2024. [Online]. Available: https://www.iso.org/standard/81870.html

[15] JFrog. (2024) End to end security tool for DevOps teams - JFrog security. Last Accessed: June 26, 2024. [Online]. Available: https://jfrog.com/xray/

[16] Joe Biden. (2021) Executive order on improving the Nation's cybersecurity. Last Accessed: June 26, 2024. [Online]. Available: https://www.whitehouse.gov/briefing-room/presidential-actions/2021/05/12/executive-order-on-improving-the-nations-cybersecurity/

[17] G. M. Kapitsaki, F. Kramer, and N. D. Tselikas, "Automating the license compatibility process in open source software with SPDX," *J. Syst. Softw.*, vol. 131, pp. 386–401, 2017.

[18] N. King, *Essential Guide to Qualitative Methods in Organizational Research*. London: Sage, 2004, ch. Using Templates in the Thematic Analysis of Text.

[19] B. Kloeg, A. Y. Ding, S. Pellegrom, and Y. Zhauniarovich, "Charting the path to SBOM adoption: A business stakeholder-centric approach," in *Procedings of Asia Conference on Computer and Communications Security*. ACM, 2024.

[20] C. Lamb and S. Zacchiroli, "Reproducible builds: Increasing the integrity of software supply chains," *IEEE Software*, vol. 39, no. 2, pp. 62–70, 2022.

[21] E. Levy, "Poisoning the software supply chain," *IEEE Security & Privacy*, vol. 1, no. 3, pp. 70–73, 2003.

[22] MSR4SBOM Team. (2024) MSR4SBOM (Mining Software Repositories for enhanced Software Bills of Materials). Last Accessed: August 3, 2024. [Online]. Available: https://msr4sbom.github.io/

[23] S. Nadgowda, "Engram: the one security platform for modern software supply chain risks," in *Proceedings of International Workshop on Container Technologies and Container Clouds*. ACM, 2022, pp. 7–12.

[24] S. Nocera, S. Romano, M. Di Penta, R. Francese, and G. Scanniello, "Software bill of materials adoption: A mining study from GitHub," in *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2023, pp. 39–49.

[25] S. Nocera, S. Romano, R. Francese, R. Burlon, and G. Scanniello, "Managing vulnerabilities in software projects: the case of NTT data," in *2023 49th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, 2023, pp. 247–253.

[26] Open Text Corporation. (2024) OpenText fortify static code analyzer — static code analysis security. Last Accessed: June 26, 2024. [Online]. Available: https://www.opentext.com/products/fortify-static-code-analyzer/

[27] OWASP. (2023) CycloneDX. Last Accessed: June 26, 2024. [Online]. Available: https://cyclonedx.org

[28] D. Paschalides and G. M. Kapitsaki, "Validate your SPDX files for open source license violations," in *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering, FSE 2016, Seattle, WA, USA, November 13-18, 2016*. ACM, 2016, pp. 1047–1051.

[29] M. Patton, *Qualitative Research & Evaluation Methods*. Sage, 2015.

[30] G. Scanniello, M. Di Penta, S. Romano, R. Francese, S. Nocera, P. Cassieri, D. Bifolco, and F. Zampetti, "MSR4SBOM: Mining software repositories for enhanced software bills of materials," in *Proceedings of the 18th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement*, ser. ESEM '24. New York, NY, USA: Association for Computing Machinery, 2024.

[31] C. Seaman, "Qualitative methods in empirical studies of software engineering," *IEEE Transactions on Software Engineering*, vol. 25, no. 4, pp. 557–572, 1999.

[32] T. Stalnaker, N. Wintersgill, O. Chaparro, M. Di Penta, D. M. German, and D. Poshyvanyk, "BOMs away! inside the minds of stakeholders: A comprehensive study of bills of materials for software systems," in *Proceedings of the IEEE/ACM International Conference on Software Engineering*. ACM, 2024.

[33] Tenable. (2024) Nessus vulnerability scanner: Network security solution. Last Accessed: June 26, 2024. [Online]. Available: https://www.tenable.com/products/nessus/

[34] The European Commission. The organic logo - European Commission. Last Accessed: August 3, 2024. [Online]. Available: https://agriculture.ec.europa.eu/farming/organic-farming/organic-logo_en

[35] The Linux Foundation. (2022) The state of software bill of materials (SBOM) and cybersecurity readiness. Last Accessed: June 26, 2024. [Online]. Available: https://www.linuxfoundation.org/research/the-state-of-software-bill-of-materials-sbom-and-cybersecurity-readiness/

[36] ——. (2023) The Software Package Data Exchange (SPDX). Last Accessed: June 26, 2024. [Online]. Available: https://spdx.dev

[37] V. A. Thurmond, "The point of triangulation," *Journal of nursing scholarship*, vol. 33, no. 3, pp. 253–258, 2001.

[38] US Department of Commerce. (2021) The minimum elements for a software bill of materials (SBOM). [Online]. Available: https://www.ntia.doc.gov/files/ntia/publications/sbom_minimum_elements_report.pdf

[39] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012.

[40] B. Xia, T. Bi, Z. Xing, Q. Lu, and L. Zhu, "An empirical study on software bill of materials: Where we stand and the road ahead," in *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 2023, pp. 2630–2642.

[41] N. Zahan, E. Lin, M. Tamanna, W. Enck, and L. Williams, "Software bills of materials are required. are we there yet?" *IEEE Security & Privacy*, vol. 21, no. 2, pp. 82–88, 2023.

Authorized licensed use limited to: University Bern. Downloaded on April 20,2026 at 14:37:43 UTC from IEEE Xplore. Restrictions apply.